2025 International Conference on Trustworthy Big Data and Artificial Intelligence (ICTBAI) | 979-8-3315-5509-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICTBAI68361.2025.00009

TOPKPHM:Periodic Patterns Mining of Top-K High Utility Itemsets

Xintian Zhang
Baotou Teachers' College of Inner Mongolia University of Science and Technology
School of Information Science and Technology
Baotou, China
1445724657@qq.com

Baowei Liu
Baotou Teachers' College of Inner Mongolia University of Science and Technology
School of Information Science and Technology
Baotou, China
bjlbw@163.com

Jing Chen*
Baotou Teachers' College of Inner Mongolia University of Science and Technology
School of Information Science and Technology
Baotou, China
* Corresponding author: chenjing@bttc.edu.cn

Shengyi Yang*
Guizhou Minzu University
School of Physics and Mechatronic Engineering
Guiyang, China
* Corresponding author: yangyshy@163.com

Zhixue Liu
Baotou Teachers' College of Inner Mongolia University of Science and Technology
School of Information Science and Technology
Baotou, China
liu-zhixue@163.com

Zhongkai Gu
Baotou Teachers' College of Inner Mongolia University of Science and Technology
School of Information Science and Technology
Baotou, China
2248221386@qq.com

Shun Xu
Inner Mongolia University of Science and Technology
School of Digital and Intelligent Industry
Baotou, China
2580198989@qq.com

***Abstract*—The goal of periodic high-utility itemsets mining (PHUIM) is to uncover item combinations with high utility over recurring intervals based on minimum utility constraints. Traditional periodic high utility itemsets mining (PHUIM) algorithms rely on preset thresholds, which are difficult to choose without prior data knowledge. This work presents Top-K periodic high utility itemsets mining (TOPKPHM), a Top-K approach that addresses threshold specification by dynamically maintaining the utility value at the K-th rank position. TOPKPHM employs the Estimated Utility & Support Co-Occurrence Structure (EUSCS) for unified utility and support management. An enhanced utility-list structure integrates periodicity constraints directly, eliminating separate data structure overhead. Extensive experiments on four real-world datasets show TOPKPHM significantly outperforms state-of-the-art algorithms, achieving execution time reductions of multiple times compared to traditional methods on the chess dataset while maintaining consistent memory efficiency. This work advances constraint-based pattern mining theory and provides practical solutions for large-scale temporal data analysis.**

Keywords-component; Data mining; Periodic pattern;Top-K utility mining

Funded Project： Industry-University-Research Innovation Fund for Chinese Universities（No. 2024HY022); Natural Science Foundation of Inner Mongolia Autonomous Region of China (No.2022MS06010,No.2025MS06044,No.QN06036), the Science and Technology Innovation Talent Team Project of Data Science and Computing Intelligence of Guizhou Province (Grant no. QKHRC-CXTD2025038); Research Project of Baotou Teachers 'College(BSJG23Z07);General Project of Natural Science for High-Level Research Achievements Cultivation Program of Baotou Teachers' College (BSY2010-26)

I. INTRODUCTION

Data mining is essential in domains such as business, healthcare, and social media, where it uncovers meaningful insights from vast data collections—especially through pattern mining, which supports informed decision-making. Gan addressed market-basket data by integrating positive and negative utilities in the PHMN+ algorithm[1]. Zhang proposed TKUS for extracting Top-K high-utility sequential patterns, showing superiority over TKHUS-Span[2]. While Top-K strategies have been used in high-utility and periodic pattern mining separately, their integration in Top-K periodic high-utility pattern mining is still emerging. For instance, the TSPIN[3] framework addresses Top-K periodic frequent patterns but ignores utility considerations. A two-stage approach of mining periodic patterns and ranking by utility faces challenges in search-space control and pruning efficiency, highlighting the need for more effective algorithms.

II. RELATED WORK

A. Top-K High Utility Mining

Recent studies on Top-KHUPM emphasize enhancing the efficiency, usability, and practical relevance of the mining process. One major trend is the proliferation of threshold-free

979-8-3315-5509-2/25/$31.00 ©2025 IEEE
DOI 10.1109/ICTBAI68361.2025.00009

algorithms that relieve users from guessing parameters—whether through Top-K frameworks[4], adaptive threshold-raising strategies [5]. Performance optimization is another key theme: new methods employ clever data structures such as global utility tables, projected databases, and utility lists, along with pruning techniques to cut down the search space and runtime dramatically[6].

In terms of novel pattern types, researchers expanded high utility pattern mining to incorporate additional dimensions—for example, handling user-specified focus items to support targeted queries[7], and leveraging item taxonomies to identify multi-level patterns. A survey examines the latest advancements in incremental High Average Utility Itemset Mining methods—such as those based on Apriori, tree structures, and utility lists—highlighting their advantages and drawbacks[8]. The incremental mining approach[9] enables continuous analytics as databases grow, which is vital for domains like e-commerce or sensor data that accumulate rapidly. Similarly, support for negative utilities and costs such as loss-leading items ensures that the patterns reflect realistic profit calculations.

B. Periodic pattern mining

Qi[10] introduces the CPR-Miner algorithm, which focuses on extracting recent high-utility patterns that are both closed and exhibit periodic characteristics, thereby emphasizing their temporal relevance and offering lossless results aligned with current trends. In response to the limitation of earlier periodic pattern mining approaches—particularly their neglect of utility and restriction to single-sequence analysis—Chen[11] proposed the MHUPFPS algorithm to recognize commonly recurring patterns with both periodic behavior and high utility value across multiple sequence datasets.

This study proposes a novel algorithm, TOPKPHM, which innovatively integrates the Top-K mining framework with periodic utility-based pattern discovery. The main contributions of this research can be summarized as follows:

· This study focuses on two crucial dimensions: discovering patterns with high utility and uncovering those with periodic characteristics. As far as we are aware, the proposed TOPKPHM algorithm is the first to integrate the Top-K mining paradigm within the context of periodic high-utility pattern discovery.

· This study introduces the EUSCS pruning method, which leverages pre-computed item-pair co-occurrence metrics (TWU/Support) to eliminate low-utility combinations early, by integrating utility upper bounds, periodic constraints, and support-based pruning—thereby greatly enhancing computational efficiency.

· The performance of TOPKPHM was validated through extensive testing on real datasets, revealing its effectiveness.

III. Problem Definition

This section introduces briefly describes key definitions related to the target high-utility itemset mining problem.We define a database D (as shown in Table 1) consisting of different items represented as $I = i_1, i_2, \dots, i_m$, where m indicates the total number of items in the set. Let $D = \{T_1, T_2, \dots, T_c, \dots, T_n\}$ represent a collection of n transactions, where each transaction in the set involves assigning a transaction ID $T_c (1 \le c \le n)$ to each item in the set.

Table 1 Example Database *D*

Tid	Transaction	TU
T_1	[*A*,3] [*B*,5] [*D*,4]	44
T_2	[*B*,3] [*D*,2] [*E*,1]	21
T_3	[*A*,2] [*B*,2] [*C*,1] [*D*,1]	18
T_4	[*C*,3] [*E*,1]	5
T_5	[*A*,4] [*B*,4] [*D*,3]	39

Table 2 Example Profit Table

Item	*A*	*B*	*C*	*D*	*E*
Profit	3	3	1	5	2

Definition 1: In transaction T_c, the utility of item X is denoted as $u(X, T_c)$, which is defined as the product of its internal utility and external utility (as shown in Table 2), $q(X, T_c) \times p(X)$. For example, $u(B, T_3) = 3 \times 2 = 6$. The utility of an item set X in transaction T_c is denoted as $u(X, T_c)$, which is defined as $u(X, T_c) = \sum_{i \in X} u(X, T_c)$. For example, the utility of the item set $\{b, e\}$ in transaction T_4 is $u(\{C, E\}, T_4) = 3 \times 1 + 1 \times 2 = 5$. For example, the utility of the itemset $\{A, B\}$ in database D is $u(\{A, B\}) = u(\{A, B\}, T_1) + u(\{A, B\}, T_3) + u(\{A, B\}, T_5) = 60$.

Definition 2: The transaction utility $TU(T_c)$ of transaction T_c refers to the total utility of transaction T_i, defined as $TU(T_c) = \sum_{X \in T_c} u(X, T_c)$. For example, $TU(T_1) = 9 + 15 + 20 = 44$.

Definition 3: Transaction weighted utility TWU(X) refers to the sum of the utilities of transactions including item set X, defined as $TWU(X) = \sum_{X \subseteq T_c \in D} TU(T_c)$. For example, the transaction weighted utility of item set $\{B, D\}$ is TWU($\{B, D\}$)=$TU(T_1)$+ $TU(T_2) + TU(T_3) + TU(T_5)$ =122.

Definition 4: Let a transaction T_c contain item X, we define the residual utility of item X in T_c as $RU(X, T_c) = \sum_{i_j \in T_c, i_j > X} (p(X) \times q(X, T_c))$. we define the residual utility of item X as $RU(X) = \sum_{X \in T_c, T_c \in D} RU(X, T_c)$. For example, assuming that $>$ is defined in alphabetical order, the residual utility RU($\{C\}$,T_3)=$u(A, T_3)$+$u(B, T_3)$+$u(D, T_3)$=6+6+5=17. The residual utility of item $\{C\}$ in database D, RU($\{C\}$)=RU($\{C\}$,T_3)+RU($\{C\}$,T_4)=17+2=19.

Definition 5: The utility of an itemset X in database D is denoted as $u(X) = \sum_{T_c \in T(X)} u(X, T_c)$, where $T(X)$ is the transaction set containing X. We define $PS(X) = \{tid_{i+1} - tid_i | i = 0,1,2, \dots, m\}$, where $tid_0 = 0$, $tid_{m+1} = |D|$. With $PS(X)$ can we get the maximum time interval, the minimum time interval and average time interval of X, which is denoted as $maxPer(X), minPer(X),$ and $avgPer(X) = (\sum_{k \in PS(X)} k) / |PS(X)| = |D| / (|Support(X)| + 1)$. For example, in Table 1, for the itemset$\{B\}$, we have the $T(B) = \{1,2,3,5\}$. Then *PS*(*B*)={1,1,1,2,1}, and we have maxPer($\{B\}$) = max{*PS*(*B*)} = 2, *minPer*($\{B\}$) = min{*PS*(*B*)} = 1, *avgPer*(*B*) = 5/(4+1) = 1.

Definition 6: We define $>$ as the total order of the itemset I. The utility-list of an item X in the database D is a set of

tuples. Each transaction T_{tid} containing item X has a corresponding triple (tid,iutil,rutil), where u(X,T_{tid}) represents the utility of item X in transaction T_{tid}, where rutil is defined as $\sum_{i\in T_{tid}\wedge i>x,\forall x\in X} u(i,T_{tid})$. The utility list of {C} is {(T_3,1,17),(T_4,3,2)}.

If the utility $u(X)$ of itemset X is not less than the minimum threshold given by the user, that is, $u(X) \geq minutil, maxper(X) \leq maxPer, avgper(X) \leq maxAvg$, then the itemset X is called a Periodic High-Utility Itemset.

IV. TOPKPHM ALGORITHM DESIGN

A distinct HUIM problem where users seek only the most significant patterns rather than all exceeding a fixed threshold. To address this, we develop efficient data structures and pruning strategies to optimize mining performance.

A. Algorithm Process

This section details the proposed algorithm, TOPKPHM, for mining Top-K periodic high-utility patterns. As illustrated in Figure 1, the framework encapsulates the entire process. Notably, TOPKPHM is capable of discovering the top-k periodic high-utility itemsets without relying on a manually set minimum utility threshold.

Using a utility-list structure within a depth-first search framework in Algorithm1, it employs pruning strategies to identify the K highest-utility patterns meeting periodic constraints. During initial database scanning, TOPKPHM preprocesses items by calculating TWU, support counts, and periodicity statistics (minimum/maximum/average intervals) through transaction ID tracking. Only candidates meeting support thresholds and max-periodicity limits are retained, sorted by ascending TWU for efficient pruning.A second scan constructs periodic utility-lists with TWU-based item ordering. This enables EUSCS optimization for co-occurrence-based pruning. Finally, the algorithm outputs results sorted by utility through its TopK manager.

Algorithm1 Top-K Periodic High-Utility Pattern Mining(TOPKPHM)

Require: Database D, parameters k, $maxPer$, $maxAvg$
Ensure: Top-K periodic high-utility itemsets
  Initialize TopKManager(k), $supThreshold \leftarrow |D|/maxAvg - 1$
  for each transaction T_{tid} in D **do**
    Update TWU[x], support[x], periodicity[x] for each item $x \in T_{tid}$
  end for
  Keep items: $Support \geq supThreshold \wedge largestPer \leq maxPer$
  for each transaction T_{tid} in D **do**
    Build $UL_x with(tid, iutil, rutil)$ for $validitems$
    Update EUSCS
  end for
  for each UL_x $where$ $UL_x.utils \geq threshold$ **do**
    if $avgPer \leq maxavg \wedge maxper \leq maxPer$ **then**
      TopKManager.add(X,utility,periodicities)
    end if
    for each UL_Y $after$ UL_X $with$ $EUSCS[X][Y] \geq threshold$ **do**
      Merge $UL_X, UL_Y \rightarrow UL_{XY}$
      Check periodicity
      if UL_{XY} valid **then**
        Recursively mine with UL_{XY}
      end if
    end for
  end for

B. Core Data Structures

1) Periodic Utility-List

To extend the functionality of the traditional utility-list, we integrate periodicity information, thereby proposing the PU-L (Periodic Utility-List Structure) in Table 3. The PU-L structure holistically encapsulates essential item characteristics, allowing for the direct derivation of critical metrics, including support, utility upper bounds, and average periodicity. The proposed periodic utility list integrates periodicity constraints directly into the utility list structure, achieving unified management of utility and periodicity information. This design eliminates the overhead of maintaining separate periodicity data structures required by traditional methods. The enhanced structure incrementally updates periodicity information during utility list construction, avoiding redundant calculations in subsequent phases. This integration enables earlier periodicity-based pruning decisions and reduces invalid candidate generation.

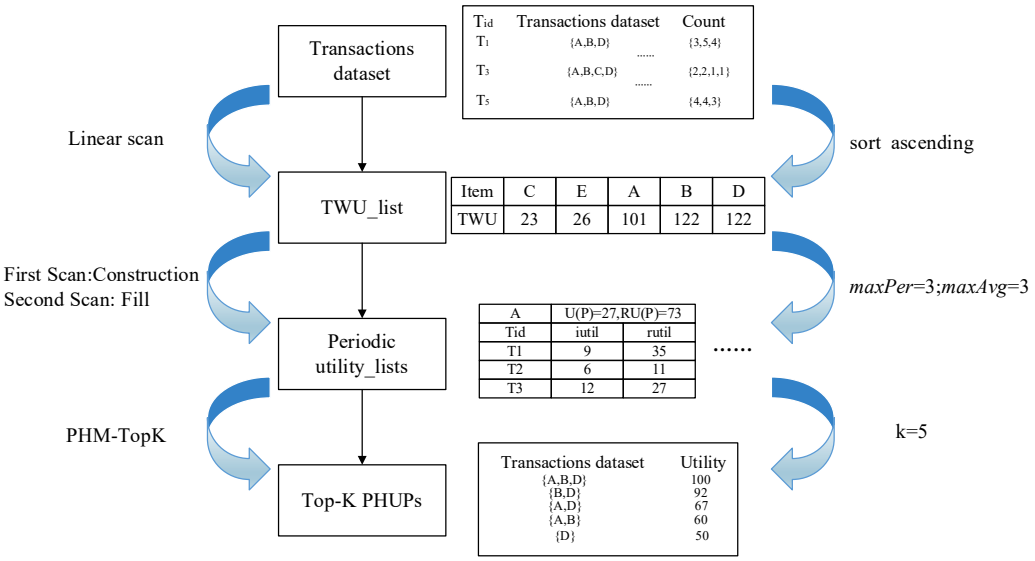

Figure 1 TOPKPHM Algorithm Framework Diagram

Table 3 Periodic Utility-List

A	U(*A*), RU(*A*), *maxPer, minPer*	
T_{tid}	*iutil*	*rutil*
T_1	9	35
T_3	6	11
T_5	12	27

The integrated structure significantly enhances performance through improved memory access locality, as related utility and periodicity information are stored within the same object, leading to better CPU cache efficiency. Computational complexity is reduced by eliminating repetitive periodicity calculations and distributing the O(n) periodicity computation across the data structure construction phase. The enhanced early pruning capability reduces processing overhead by enabling periodicity-based filtering during utility list construction rather than during candidate evaluation. These optimizations collectively improve both time and space efficiency while maintaining the completeness of pattern discovery.

2) EUSCS

Philippe Fournier-Viger[12] put forward a structure called EUCS(Estimated Utility Co-occurrence Structure) as shown in Table 4 to store the TWU of all pairs of items occuring in the database, and this structure is used to pruning any itemset P_{xy} containing a pair of items $\{x, y\}$ having a TWU lower than *minutil.* This design causes redundant storage of identical item pair keys across two hash tables. Each pruning decision requires two independent hash lookups, resulting in both space redundancy and increased time overhead.

Table 4 EUCS

	E	A	B	D
E	/	TWU({EA})	TWU({EB})	TWU({ED})
A	/	/	TWU({AB})	TWU({AD})
B	/	/	/	TWU({BD})
D	/	/	/	/

The proposed EUSCS (Estimated Utility & Support Co-occurrence Structure) is an improved version of the traditional EUCS table. Inspired by the EUCS, the EUSCS(as shown in Table 5) extends this by additionally incorporating support information and a boolean indicator. This boolean value denotes whether the TWU of each item pair exceeds the predefined minimum utility threshold(*minutil*). By integrating utility and support constraints into a unified data structure, EUSCS enables more precise and early-stage pruning, significantly reducing the generation of unpromising candidate itemsets and computational overhead during the mining process. The unified structure eliminates redundant key storage and saves approximately 50% of key space overhead, while enabling multi-dimensional pruning information retrieval through single queries that optimize two hash lookups into one. Furthermore, EUSCS improves data locality and enhances CPU cache hit rates, providing a new design approach for pruning optimization in high-utility pattern mining.

Table 5 EUSCS

	A	B	D
E	(0,0)	(1,0)	(1,0)
A	/	(3,1)	(3,1)
B	/	/	(4,1)

3) TopKManager

In terms of dynamic threshold management, the algorithm designs an adaptive estimated threshold mechanism. Through the estimated Threshold dynamic update strategy in TopKManager, it not only maintains the current minimum utility threshold, but also estimates the possible threshold change trend in the future based on the distribution characteristics of the discovered patterns.

C. Experiment

The experimental dataset specifications are detailed in Table 6, encompassing three key metrics: Transaction count (total transactions per dataset), Item count (number of distinct item types), and Avg_Trans_len (mean length of transactions). All algorithmic implementations were developed in Java. The computational environment for experimentation comprised a system featuring 16GB RAM, Intel(R) Core(TM) i7-10700 CPU, and 64-bit Windows 10 operating system. Multiple real-world datasets sourced from the SPMF data mining library were employed to conduct comprehensive evaluations of the TOPKPHM algorithm's effectiveness and scalability properties.

In the experiment, PHM was run on each dataset with fixed minper and minAvg values, while varying the minutil threshold and the values of the maxAvg and maxPer parameters. In these experiments, the values for the periodicity thresholds have been found empirically for each dataset(as they are dataset specific), and were chosen to show the trade-off between the number of periodic patterns found than the other parameters. Each algorithm uses the same parameters to ensure the fairness and reliability of the experiment. Thereafter, the notation Chess *V-W-X-Y* represents the Chess dataset with *minPer* = *V*, *maxPer* = *W*, *minAvg* = *X*, and *maxAvg* = *Y*.

Table 6 Dataset Characteristics

Dataset	Transaction count	Item count	Avg_Trans_len	*V-W-X-Y*
Chess	3196	75	37	1-100-5-100
Accidents	340183	468	33	1-20-5-20
Mushrooms	8416	119	23	1-2000-5-1000
Retail	88162	16470	1030	1-1000-5-500

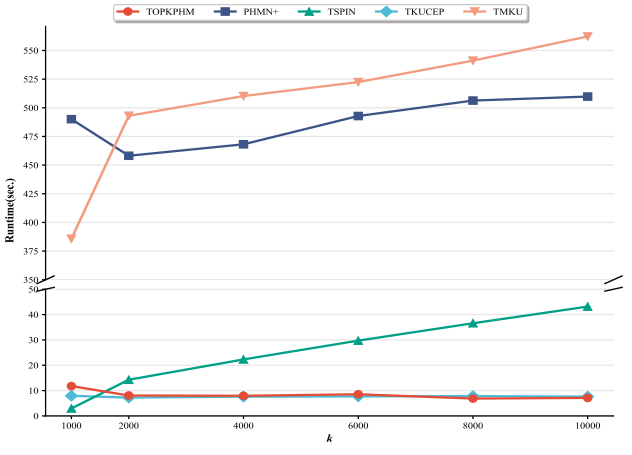

Figure 2 Running Time on the Chess Dataset

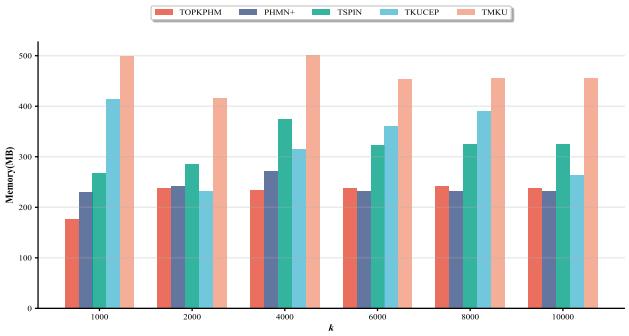

Figure 3 Memory Usage on the Chess Dataset

Figure 2 show that the TOPKPHM algorithm has obvious performance advantages on this dataset, its memory usage is relatively stable, and its overall time efficiency is higher than that of other algorithms except TKUCEP.In contrast, the TSPIN algorithm shows good time performance, good scalability, and its memory usage in figure 2 is also controlled within a reasonable range; the execution time of the TKUCEP algorithm is almost unaffected by the change of the K value, but the running memory is not small; the PHMN+ algorithm and the TMKU algorithm have too long running time, and their running time tends to increase as the k value increases.

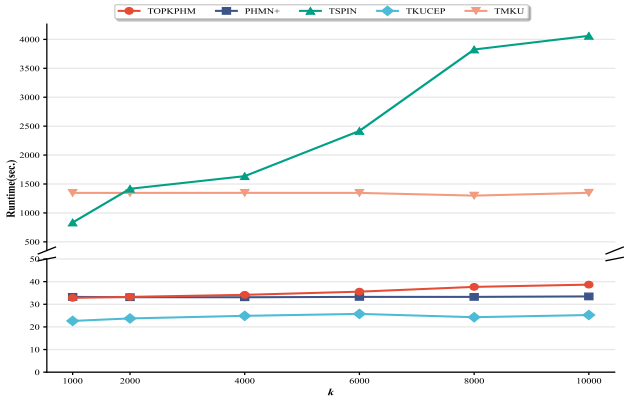

Figure 4 Running Time of Each Algorithm on the Accidents Dataset

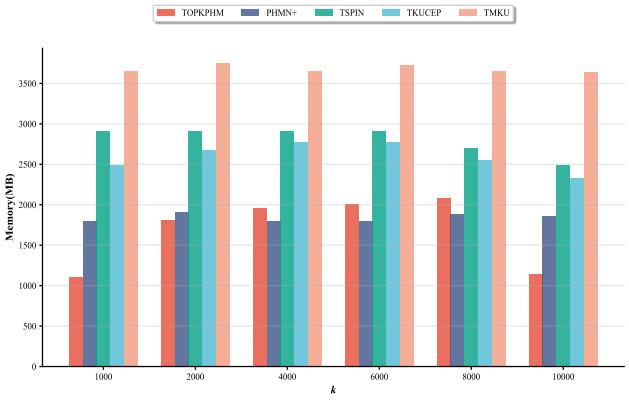

Figure 5 Memory usage Accidents Dataset

Figures 4 and 5 demonstrate the efficiency of TOPKPHM, which benefits from multi-level pruning and adaptive threshold adjustment. This allows it to outperform other methods in runtime and memory usage as k increases. TKUCEP shows instability due to fluctuating thresholds, and TMKU performs the worst due to its exhaustive search strategy. In terms of memory, TOPKPHM maintains a low footprint, while TKUCEP and TMKU consume significantly more resources.

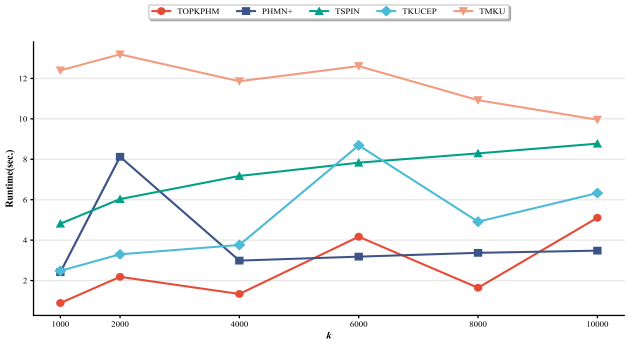

Figure 6 Running Time on the Mushrooms Dataset

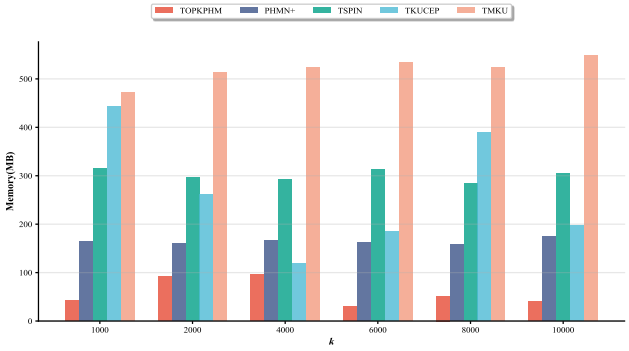

Figure 7 Memory Usage on the Mushrooms Dataset

On the Mushroom dataset(as shown in Figure 6 and Figure 7), TOPKPHM shows consistently strong performance with stable and low runtime and memory usage. Although runtime varies at some k values, it is still much faster than other methods. Memory use is efficient, ranging from 30MB to 96MB, much lower than competitors. PHMN+ has more variable runtime, with peaks at k=2000 (8124ms) and k=6000 (3372ms), and memory use between 159MB and 175MB. TSPIN consumes the most resources, with runtime over 5100ms and memory above 300MB. Traditional methods TKUCEP and TMKU perform worse.

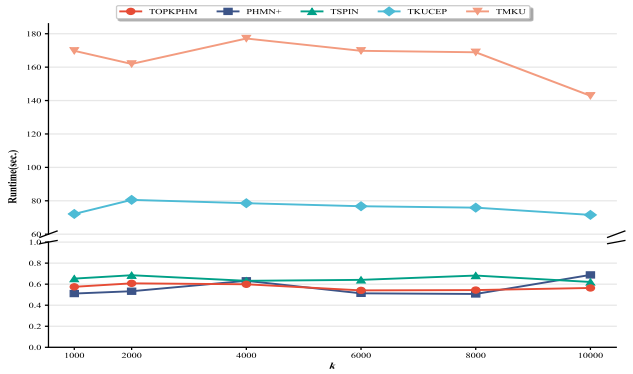

Figure 8 Running Time on the Retail Dataset

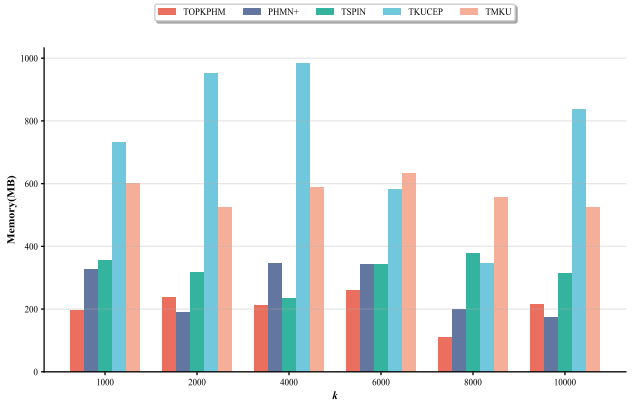

Figure 9 Memory Usage on the Retail Dataset

As shown in Figure 8 and Figure 9, the TOPKPHM algorithm, PHMN+ algorithm, and TSPIN algorithm all performed well in terms of running time, but TOPKPHM showed excellent time and space efficiency through multi-level pruning and adaptive threshold management, and maintained a stable memory usage rate. traditional methods such as TMKU have the worst performance due to imperfect pruning mechanisms. Based on the Retail dataset results, TOPKPHM demonstrates exceptional performance advantages with remarkable time efficiency improvements of up to 99.7% compared to TMKU and 99.2% compared to TKUCEP, while maintaining consistent execution times ranging from 541ms to 608ms across different k values. Unlike competing algorithms that exhibit significant performance fluctuations, TOPKPHM shows predictable memory usage between 196.7MB and 261.31MB, making it highly suitable for practical applications requiring reliable resource planning.

Despite promising results, TOPKPHM has several limitations. First, while EUSCS improves memory efficiency, the algorithm may struggle with extremely large-scale datasets due to substantial memory requirements for utility-list structures. Second, performance is sensitive to utility value distributions and periodicity patterns, potentially degrading on datasets with highly skewed utilities or irregular temporal patterns. TOPKPHM requires careful parameter tuning for periodicity constraints, which demands domain expertise and preliminary data analysis. The algorithm assumes static datasets and cannot handle streaming environments where transactions arrive continuously.

V. Conclusions

This paper addressed the threshold specification problem in periodic high-utility itemset mining by proposing TOPKPHM, a novel Top-K approach that eliminates the need for preset minimum utility thresholds. Experimental results on four datasets show TOPKPHM's superior efficiency: reducing runtime by up to 99.7% vs. TMKU and 99.2% vs. TKUCEP on Retail data, while maintaining stable execution (541-608ms). Memory usage remained low (30-261MB), outperforming competitors like TSPIN (>300MB) and PHMN+ (159-175MB). Key innovations include EUSCS pruning (50% fewer hash operations) and optimized utility-lists, enabling scalable Top-K pattern mining. These findings advance constraint-based pattern mining theory and provide practical solutions for temporal data analysis applications. However, limitations include sensitivity to data distribution characteristics and challenges with streaming environments. Future research directions include developing distributed versions for big data scalability, extending to streaming contexts, and incorporating automatic parameter optimization mechanisms to enhance the algorithm's robustness and applicability.

Acknowledgment

This work was supported by the Industry-University-Research Innovation Fund of Chinese Universities (Grant No. 2024HY022), Natural Science Foundation of Inner Mongolia Autonomous Region (Grants No. 2022MS06010, No. 2025MS06044, No. QN06036), the Science and Technology Innovation Talent Team Project of Data Science and Computing Intelligence of Guizhou Province (Grant no. QKHRC-CXTD2025038),and Research Fund of Baotou Teachers College, Inner Mongolia University of Science & Technology (Grant No. BSJG23Z07),General Project of Natural Science for High-Level Research Achievements Cultivation Program of Baotou Teachers' College (BSY2010-26).

References

[1] W. Gan, J. C. W. Lin, P. Fournier-Viger, and H. C. Chao, “Mining periodic high-utility itemsets with both positive and negative utilities,” IEEE Trans. Knowl. Data Eng., vol. 35, no. 4, pp. 3745-3759, Apr. 2023.

[2] Y. Zhang, P. Fournier-Viger, A. Fujita, and T. Murakami, “TKUS: Mining Top-K high utility sequential patterns,” in Proc. Int. Conf. Web Intelligence and Mining (WIM), Tokyo, Japan, 2021, pp. 154-163.

[3] P. Fournier-Viger, Y. Wang, P. Yang, et al., “TSPIN: mining Top-K stable periodic patterns,” Appl. Intell., vol. 52, pp. 6917-6938, Apr. 2022.

[4] R. Kumar and K. Singh, "Top-K high utility itemset mining: current status and future directions," Knowl. Eng. Rev., vol. 39, p. e5, 2024.

[5] V. V. Vu, M. T. H. Lam, T. T. M. Duong, et al., "Ftkhuim: a fast and efficient method for mining Top-K high-utility itemsets," IEEE Access, vol. 11, pp. 104789-104805, 2023.

[6] K. Singh and B. Biswas, "Mining Top-K high on-shelf utility itemsets using novel threshold raising strategies," ACM Trans. Knowl. Discov. Data, vol. 18, no. 5, pp. 1-23, 2024.

[7] S. Huang, W. Gan, J. Miao, et al., "Targeted mining of Top-K high utility itemsets," Eng. Appl. Artif. Intell., vol. 126, p. 107047, 2023.

[8] J. Chen, S. Yang, W. Ding, et al., "Incremental high average-utility itemset mining: survey and challenges," Sci. Rep., vol. 14, no. 1, p. 9924, 2024.

[9] C. Lee, H. Kim, M. Cho, H. Kim, B. Vo, J. C. W. Lin, P. Fournier-Viger, and U. Yun, " Incremental Top-K high utility pattern mining and analyzing over the entire accumulated dynamic database," IEEE Access, vol. 12, pp. 77605-77620, 2024.

[10] Y. Qi, X. Zhang, G. Chen, et al., "Mining periodic trends via closed high utility patterns," Expert Syst. Appl., vol. 228, p. 120356, 2023.

[11] C. M. Chen, Z. Zhang, J. M.-T. Wu, et al., "High utility periodic frequent pattern mining in multiple sequences," Comput. Model. Eng. Sci., vol. 137, no. 1, p. 733, 2023.

[12] P. Fournier-Viger, J. C. W. Lin, Q. H. Duong, et al., "PHM: mining periodic high-utility itemsets," in Proc. 16th Industrial Conf. Data Mining (ICDM), New York, NY, USA, 2016, pp. 64-79.